



Volume Number: 13 (1997)
Issue Number: 11
Column Tag: Programmer's Challenge

by Bob Boonstra, Westford, MA

Pente®

Reaching once again into the closet where we store board games, I found the game of Pente®, The board physically resembles the board used in GO, but the game strategies are simpler. Pente® is played by two players who alternate placing stones on a 19x19 grid. The objective is to win the game by getting five or more stones in a row or, alternatively, by capturing five or more pairs of your opponent's stones. Your Challenge is to write code that will play the game of Pente® and accumulate the most points (described below) in the minimum time. The prototype for the code you should write is:

```
typedef struct Capture {
    Point stone1;
    Point stone2;
} Capture;

void InitPente(
    long boardHalfSize          /* e.g., 9 for a 19x19 board */
                                /* all coordinates between -boardHalfSize
                                and +boardHalfSize */
);

void Pente(
    Point opponentsMove,        /* your opponent moved here */
    Boolean playingFirst,       /* ignore opponentMove */
    Point *yourMove,            /* return your move here */
    Capture claimCaptures[],    /* return coordinates of captured pairs here */
    long *numCaptures,          /* return number of claimCaptures here */
    Boolean *claimVictory       /* return true if you claim victory with this move */
);
```

```
void TermPente(void);          /* deallocate any dynamic storage */
```

Captures take place by bracketing two adjacent stones of your opponents. Given the position

```
---BWW---
```

... Black can capture the two White stones by playing ...

```
---BWWB---
```

... after which the two White stones are removed ...

```
---B B---
```

Captures can occur horizontally, vertically, or diagonally. Note that no capture occurs if White moves into the unoccupied square below:

```
---BW-B---
```

Multiple captures can occur on a single move. The game ends when one side captures five pairs of the opponent's stones, or when one side places five stones in a straight line, either horizontally, vertically, or diagonally. When one side obtains an unblocked row of four stones, called a tessera, a win is imminent. Therefore, an unblocked row of three stones, called a tria, is a serious threat that should be blocked unless a stronger offensive move exists. To

Showcase

ChronoSync 4.8



Synchronize & Backup Like Clockwork

THIS MONTH'S ISSUE



Issue 353

Code Debugging and
Spelunking

Check out the latest including:
 x86 Code Debugging and
 Spelunking ; An Introduction
 to WebGL ; Managing ESXi
 from your Mac ; Deploying
 SHA-3 ; An introduction to
 Julia ; KoolTools: BBEEdit 11.5,
 TextSoap For OS X, Honeywell
 Lyric Round Wi-Fi Thermostat,
 My Cloud EX2 Ultra ; MacTech
 Spotlight: Daniel H Steinberg

See Now



Remember
when typing
felt good?



tactilepro™
the satisfying click of
mechanical keyswitches



Matias
Folding Keyboard™
full-size & portable keyboard
for iPhone, iPad, & Mac

matias

TODAY'S DEAL

neutralize the advantage that the first player has, the first player's second move is restricted to be three or more intersections away from the center of the board (i.e., the h and v coordinates of White's second move must both be greater than or equal to 3 in absolute value). At the start of the game, your `InitPente` routine (and that of your opponent) will be called with the half-width of the board in `boardHalfSize` (between 9 and 15, inclusive). The `Pente` routines will then be alternately called, providing the location of your opponentsMove (unless you are playingFirst). You should place a stone in an unoccupied location and return that location in yourMove. In addition, if your move captures any adjacent pairs of your opponent's stones, you should report the number of captures in `numCaptures`, and the locations of the captured pairs in `claimCaptures`. If your move results in victory, either by achieving 5 of your stones in a row, or a cumulative capture of 5 pairs of your opponent's stones, you should set `claimVictory` to true. At the end of the game, `TermPente` will be called, where you should deallocate any dynamically allocated storage. Board coordinates are expressed as distance from the center square in ordered (v,h) pairs, so that the center intersection is at (0,0), and the corners of the standard board are at (-9,-9), (-9,9), etc. At the end of the game, points will be awarded as follows: 5 points for the player who achieved 5 stones in a row 1 point for each capture 1 point for each distinct sequence of 4 stones in a row remaining on the board One point will be deducted for each second of execution time used during a player's turns, including the time taken by the `InitPente` and `TermPente` routines. Both the game winner and the game loser will accumulate points. It is possible to earn negative points. The Challenge winner will be the entry that accumulates the most points in a round-robin tournament of all entries, where each entry plays each other entry at least twice, half of the time playing first and half playing second. Your code may allocate up to 10MB of dynamic storage, including both explicit calls to `NewPtr/malloc` and allocation of dynamic objects. This will be a native PowerPC Challenge, using the latest `CodeWarrior` environment. Solutions may be coded in C, C++, or Pascal. `Pente®` is published by Pente Games, Inc.

Three Months Ago Winner

Congratulations to Peter Lewis (Perth, Australia) for submitting the winning entry to the Stratego Challenge. Peter's entry was a convincing winner in a tournament of the 5 entries submitted. The Stratego tournament consisted of 80 games, with each entry playing against each other entry 8 times, 4 times playing first and 4 times playing second. Peter's entry plays a very good game of Stratego, as evidenced by the fact that it won 30 of the 32 games it played. His algorithm, described in the commentary at the beginning of his code, includes aggressively attempting to capture weaker pieces, exploring with low ranking pieces of his own, running away from stronger pieces, exchanging pieces of equal rank, and advancing toward unknown pieces. Of the 30 games Peter's entry won, 10 were won by capturing the opponent's flag, and 19 were won by eliminating all of the pieces that the opponent could move. (The remaining win resulted from resignation of the opponent.) Several of the entries used delaying tactics in an attempt to take advantage of the scoring rules and force their opponent to earn negative points for a win. As discussed on the Challenge mailing list, I cut off games where one player took longer than 20 seconds, declared a tie, and awarded points (sometimes negative points) to each player. The table below lists the tournament results and point totals for each entry. The number in parentheses after the entrant's name is the total number of Challenge points earned in all Challenges to date prior to this one.

Name	Wins	Points	Code	Data	Language
Peter Lewis (37)	30	296.62	13436	310	C++
Dennis Jones	13	86.75	8632	414	C
Randy Boring (39)	5	58.68	5724	618	C
Ernst Munter (286)	9	29.45	12044	4072	C++
Tom Saxton (10)	7	-51.69	7320	460	C

Top 20 Contestants

Here are the Top Contestants for the Programmer's Challenge. The numbers below include points awarded over the 24 most recent contests, including points earned by this month's entrants.

Rank	Name	Points	Rank	Name	Points
1.	Munter, Ernst	200	11.	Antoniewicz, Andy	24
2.	Gregg, Xan	63	12.	Picao, Miguel Cruz	21
3.	Lewis, Peter	57	13.	Day, Mark	20
4.	Cooper, Greg	54	14.	Higgins, Charles	20
5.	Boring, Randy	41	15.	Studer, Thomas	20
6.	Lengyel, Eric	40	16.	Saxton, Tom	17
7.	Mallett, Jeff	30	17.	Gundrum, Eric	15
8.	Murphy, ACC	30	18.	Hart, Alan	14
9.	Nicolle, Ludovic	28	19.	O'Connor, Turlough	14
10.	Larsson, Gustav	27	20.	Karsh, Bill	12

There are three ways to earn points: (1) scoring in the top 5 of any Challenge, (2) being the first person to find a bug in a published winning solution or, (3) being the first person to

CAPE AND TIGHTS NOT INCLUDED

Switch from reactive to **PROACTIVE** support and be the superhero your users expect

WATCHMAN
MONITORING



Search

Community Search:

GO

MacTech Search:

GO

Software Updates via MacUpdate

Remember
when typing
felt good?



Latest Forum Discussions

SEE ALL

iPHONE APP SHOWCASE

Price Scanner via MacPrices.net

Jobs Board

suggest a Challenge that I use. The points you can win are:

1st place	20 points	5th place	2 points
2nd place	10 points	finding bug	2 points
3rd place	7 points	suggesting Challenge	2 points
4th place	4 points		

Here is Peter's winning solution:

Challenge.cp

© 1997 Peter N Lewis

```
#define ASSERTIONS 0
#define DEBUG_RULES 0

#include Timer.h
#include stdlib.h
#include string.h

#include "Challenge.h"

/*
Author: Peter N Lewis

Assumptions:
Only time we spend thinking is counted against out 10 seconds (not time
in GetMove/ReportMove)
[Actually, this assumption is not valid, but Peter won anyway. --Bob]

Method:
Basically we keep track of the board and what we know and what they know.
Each opponent piece has a bit map associated with it describing what pieces
it could be. As we see more pieces, the bit map is culled. If the piece
moves, the bomb & flag bits are removed. If we've seen all Scouts
(for example), then the Scout bit is removed from all remaining pieces. If
all but one bit is removed, then we know what the piece is.

At each turn, we simply apply a sequence of actions (listed below) and
take the first action that works. It does very little in the way of
lookahead (it plans out a path, but doesn't remember it and doesn't take
it to account any movement by the opposition)

It keeps a CRC of recent board positions (since the last strike) and doesn't
replay any boards (we want to win, not draw!).

If we exceed 10 seconds thinking time, we resign. Not that this is particularly
likely, in the games I tried, it spend less than half a second total.

Optimizations:
None.

Comment:
It actually plays a half decent game! The end game is not as good as I'd
like, but time is up!
*/

/*
USE SPY
If our spy is next to their 1, kill it

DEFEND AGAINST SPY
if we have seen the spy, ignore this case

If an unknown piece is next to the 1, then run, attack, have another piece attack,
or ignore depending on a table

ATTACK WEAKER
If a known piece is next to a weaker known piece, attack it except if it places
that piece in a dangerous location

EXPLORE ATTACK
If a 6,7,9 is next to an unknown piece, attack it
```

RETREAT

If a known piece is next to a stronger known piece, run away (preferably towards something that can kill it or if it's lowly, towards an unknown piece)

SCOUT

Try advancing scouts rapidly

ATTACK DISTANT

If a known piece is distant, but a clear path leads a slightly better piece towards it, advance the better piece (includes miners)

EXPLORE DISTANT

Try exploring (advance lowly pieces towards unknown pieces)

ATTACK KNOWN WITH SAME DISTANT

If a known piece can be attacked by a known identical piece, attack it

FIND FLAG

When few unmoved pieces remain, start assuming they are bombs/flags

MOVE FORWARD

Move any piece we can forward

MOVE

Move any piece we can

RESIGN

Give up

*/

#if ASSERTIONS

static void Assert(short must)

```
{
    if ( !must ) {
        DebugStr( "\\pAssert failed!\\n" );
    }
}
```

#else

#define Assert(must)

#endif

enum {

```
    kEmpty = kFlag+1,
    kWater,
    kMoved,           // fake rank for moved pieces
    kAddForRankish // add this in for enemies when calculating the CRC
};
```

enum {

```
    kNoColor = 0
};
```

enum {

```
    kNothing = 0x00001FFE,
    kStationaryBits = ((1 << kBomb) | (1 << kFlag))
};
```

enum {

```
    kRepeatedBoards = 1000
};
```

typedef struct Square {

```
    PlayerColor color;
    PieceRank rank;
    UInt32 possibilities;
} Square;
```

typedef Square OurBoard[kBoardSize][kBoardSize];

```

typedef int Counts[kFlag+1];

typedef UInt32 BoardPossibilities[kBoardSize][kBoardSize];

typedef struct Storage {
    UInt32 total_time;
    UInt32 extra_time;
    OurBoard board;
    Counts our_pieces;
    Counts their_pieces;
    Boolean do_getmove;
    Boolean victory;
    Square blankSquare;
    PlayerColor playerColor;
    PlayerColor theirColor;
    BoardPossibilities dangers;
    BoardPossibilities known_dangers;
    UInt32 repeated_board[kRepeatedBoards];
    UInt32 repeated_board_count;
} Storage, *StoragePtr;

static char *board_setup[4] = {
// 1 = Marshal, ..., 9 = Scout, : = Spy, ; = Bomb, < = Flag
    "8;<;67;7;7",
    "48;3862;89",
    "6359954865",
    "997159:499",
};

static char *start_piece_counts = "0112344458161";

static int dR[4] = { 1, 0, -1, 0 };
static int dC[4] = { 0, -1, 0, 1 };

#if ASSERTIONS
AssertValidBoard
static void AssertValidBoard( StoragePtr storage )
{
    int piece;
    int count1 = 0;
    int count2 = 0;
    int row, col;

    for ( piece = kMarshall; piece <= kFlag; piece++ ) {
        count1 += storage->their_pieces[piece];
    }

    for ( row = 0; row < kBoardSize; row++ ) {
        for( col = 0; col < kBoardSize; col++ ) {
            if ( storage->board[row][col].color == storage->theirColor
                && storage->board[row][col].rank == kUnknown ) {
                count2++;
            }
        }
    }

    Assert( count1 == count2 );
}

#else

#define AssertValidBoard( storage )

#endif

PositionPieces
void PositionPieces(
    void *privStorage,          /* 1MB of preinitialized storage for your use */
    PlayerColor playerColor,    /* you play red or blue, with red playing first */
    Board *theBoard             /* provide the initial position of your pieces */
)
{

```

```

StoragePtr storage = (StoragePtr) privStorage;
int row, our_row, their_row, col, board_col;
PlayerColor theirColor;
int piece;
Boolean reverse = (TickCount() & 1) != 0;

Assert( strlen(board_setup[0]) == kBoardSize );
Assert( strlen(board_setup[1]) == kBoardSize );
Assert( strlen(board_setup[2]) == kBoardSize );
Assert( strlen(board_setup[3]) == kBoardSize );

for ( row = 0; row <= 3; row++ ) {
    if ( playerColor == kRed ) {
        our_row = row;
        their_row = (kBoardSize-1)-row;
        theirColor = kBlue;
    } else {
        their_row = row;
        our_row = (kBoardSize-1)-row;
        theirColor = kRed;
    }
    for ( col = 0; col < 10; col++ ) {
        board_col = reverse ? (kBoardSize-1) - col : col;
        (*theBoard)[our_row][col].thePieceRank = (PieceRank)
            (board_setup[row][board_col] - '0');
        (*theBoard)[our_row][col].thePieceColor = playerColor;

        storage->board[our_row][col].color = playerColor;
        storage->board[our_row][col].rank =
            (*theBoard)[our_row][col].thePieceRank;
        storage->board[our_row][col].possibilities = kNoNothing;

        storage->board[their_row][col].color = theirColor;
        storage->board[their_row][col].rank = kUnknown;
        storage->board[their_row][col].possibilities = kNoNothing;
    }
}

for ( row = 4; row <= 5; row++ ) {
    for( col = 0; col < kBoardSize; col++ ) {
        storage->board[row][col].color = (PlayerColor)kNoColor;
        storage->board[row][col].rank =
            (PieceRank) ((col/2 % 2 == 1) ? kWater : kEmpty);
        storage->board[row][col].possibilities = 0;
    }
}

for ( piece = kMarshall; piece <= kFlag; piece++ ) {
    storage->our_pieces[piece] =
        start_piece_counts[piece] - '0';
    storage->their_pieces[piece] =
        start_piece_counts[piece] - '0';
}

storage->do_getmove = (playerColor == kBlue);
storage->victory = false;
storage->blankSquare = storage->board[4][0];
storage->playerColor = playerColor;
storage->theirColor = playerColor == kRed ? kBlue : kRed;
storage->repeated_board_count = 0;

AssertValidBoard( storage );
}

Learn
static void Learn( StoragePtr storage, Boolean them,
                  int row, int col, PieceRank rank )
{
    Boolean gotall;
    PlayerColor thiscolor;
    int r, c;

    if ( storage->board[row][col].rank == kUnknown ) {

```

```

if ( rank == kMoved ) {
    UInt32 possibilities =
        storage->board[row][col].possibilities;
    possibilities &= ~kStationaryBits;

    if ( (possibilities & (possibilities-1)) == 0 ) {
        // only one bit on! Now we know!
        int newrank;
        newrank = 0;
        while ( (possibilities & 1) == 0 ) {
            possibilities >>= 1;
            newrank++;
        }
        rank = (PieceRank)newrank;
    } else {
        storage->board[row][col].possibilities = possibilities;
    }
}

if ( rank != kMoved ) {
    storage->board[row][col].rank = rank;
    storage->board[row][col].possibilities = (1 << rank);
    if ( them ) {
        gotall = --storage->their_pieces[rank] == 0;
    } else {
        gotall = --storage->our_pieces[rank] == 0;
    }
    if ( gotall ) {
        thiscolor = storage->board[row][col].color;
        for ( r = 0; r < kBoardSize; r++ ) {
            for ( c = 0; c < kBoardSize; c++ ) {
                if ( storage->board[r][c].rank == kUnknown
                    && storage->board[r][c].color == thiscolor ) {
                    UInt32 possibilities =
                        storage->board[r][c].possibilities;
                    possibilities &= ~ (1 << rank);
                    storage->board[r][c].possibilities = possibilities;
                    if ( (possibilities & (possibilities-1)) == 0 ) {
                        // only one bit on!
                        int newrank;
                        newrank = 0;
                        while ( (possibilities & 1) == 0 ) {
                            possibilities >>= 1;
                            newrank++;
                        }
                        Learn( storage, them, r, c, (PieceRank)newrank );
                    }
                }
            }
        }
    }
} else {
    Assert( rank == kMoved ||
        storage->board[row][col].rank == rank );
}
}

```

HandleTheirMove

```

static void HandleTheirMove( StoragePtr storage,
    GetOpponentMove GetMove )
{
    PiecePosition moveFrom;
    PiecePosition moveTo;
    Boolean moveStrike;
    MoveResult moveResult;
    UnsignedWide start, finish;

    Microseconds( &start );
    (*GetMove)( &moveFrom,&moveTo, &moveStrike,&moveResult );
    Microseconds( &finish );
    storage->extra_time += finish.lo - start.lo;
}

```

```

Assert( moveResult.legalMove );
// They must have made a legal move or we would not be called
Assert( !moveResult.victory );
// If they won we would not be called
if ( moveStrike ) {
    Learn( storage, true, moveFrom.row, moveFrom.col,
           moveResult.rankOfAttacker.thePieceRank );
    Learn( storage, false, moveTo.row, moveTo.col,
           moveResult.rankOfDefender.thePieceRank );
    if ( moveResult.attackerRemoved &&
          moveResult.defenderRemoved ) {
        storage->board[moveFrom.row][moveFrom.col] =
            storage->blankSquare;
        storage->board[moveTo.row][moveTo.col] =
            storage->blankSquare;
    } else if ( moveResult.attackerRemoved ) {
        if ( storage->board[moveTo.row][moveTo.col].rank == kBomb ) {
            storage->board[moveFrom.row][moveFrom.col] =
                storage->blankSquare;
        } else {
            storage->board[moveFrom.row][moveFrom.col] =
                storage->board[moveTo.row][moveTo.col];
            storage->board[moveTo.row][moveTo.col] =
                storage->blankSquare;
        }
    } else {
        Assert( moveResult.defenderRemoved );
        storage->board[moveTo.row][moveTo.col] =
            storage->board[moveFrom.row][moveFrom.col];
        storage->board[moveFrom.row][moveFrom.col] =
            storage->blankSquare;
    }
} else {
    storage->board[moveTo.row][moveTo.col] =
        storage->board[moveFrom.row][moveFrom.col];
    storage->board[moveFrom.row][moveFrom.col] =
        storage->blankSquare;
    if ( abs(moveTo.row - moveFrom.row) +
          abs(moveTo.col - moveFrom.col) > 1 ) {
        Learn( storage, true, moveTo.row, moveTo.col, kScout );
    } else {
        Learn( storage, true, moveTo.row, moveTo.col,
              (PieceRank)kMoved );
    }
}
}

AssertValidBoard( storage );
}

FindPiece
static Boolean FindPiece( StoragePtr storage, PlayerColor color,
                        PieceRank rank, int *row, int *col )
{
    int r, c;

    for ( r = 0; r < kBoardSize; r++ ) {
        for( c = 0; c < kBoardSize; c++ ) {
            if ( storage->board[r][c].color == color
                && storage->board[r][c].rank == rank ) {
                *row = r;
                *col = c;
                return true;
            }
        }
    }
    return false;
}

IsOnBoardWeak
static Boolean IsOnBoardWeak( int row, int col )
{

```

```

    return 0 <= row && row < kBoardSize &&
           0 <= col && col < kBoardSize;
}

IsOnBoard
static Boolean IsOnBoard( int row, int col )
{
    if ( 0 <= row && row < kBoardSize &&
         0 <= col && col < kBoardSize ) {
        if ( row <= 3 || row >= 6 ) {
            return true;
        }
        if ( col <= 1 || col >= 8 ) {
            return true;
        }
        if ( 4 <= col && col <= 5 ) {
            return true;
        }
    }
    return false;
}

IsColorPiece
static Boolean IsColorPiece( StoragePtr storage,
                             int row, int col, PlayerColor color )
{
    Assert( IsOnBoardWeak( row, col ) );
    return storage->board[row][col].color == color;
}

IsOurPiece
static Boolean IsOurPiece( StoragePtr storage, int row, int col )
{
    Assert( IsOnBoardWeak( row, col ) );
    return storage->board[row][col].color == storage->playerColor;
}

IsTheirPiece
static Boolean IsTheirPiece( StoragePtr storage,
                             int row, int col )
{
    Assert( IsOnBoardWeak( row, col ) );
    return storage->board[row][col].color == storage->theirColor;
}

IsUnknownPiece
static Boolean IsUnknownPiece( StoragePtr storage,
                               int row, int col )
{
    Assert( IsOnBoardWeak( row, col ) );
    return storage->board[row][col].rank == kUnknown;
}

IsRankPiece
static Boolean IsRankPiece( StoragePtr storage,
                            int row, int col, PieceRank rank )
{
    Assert( IsOnBoardWeak( row, col ) );
    return storage->board[row][col].rank == rank;
}

IsEmptySquare
static Boolean IsEmptySquare( StoragePtr storage,
                              int row, int col )
{
    Assert( IsOnBoardWeak( row, col ) );
    return storage->board[row][col].rank == (PieceRank)kEmpty;
}

IsWaterSquare
static Boolean IsWaterSquare( StoragePtr storage,
                              int row, int col )
{

```

```

    Assert( IsOnBoardWeak( row, col ) );
    return storage->board[row][col].rank == (PieceRank)kWater;
}

IsLowlyRank
static Boolean IsLowlyRank( PieceRank rank )
{
    return kCaptain <= rank && rank <= kScout && rank != kMiner;
}

IsLowlyPiece
static Boolean IsLowlyPiece( StoragePtr storage,
                           int row, int col )
{
    Assert( IsOnBoard( row, col ) );
    return IsLowlyRank( storage->board[row][col].rank );
}

IsMovedPiece
static Boolean IsMovedPiece( StoragePtr storage,
                           int row, int col )
{
    Assert( IsOnBoard( row, col ) );
    return (storage->board[row][col].possibilities &
            kStationaryBits) == 0;
}

IsRevealedPiece
static Boolean IsRevealedPiece( StoragePtr storage,
                              int row, int col )
{
    Assert( IsOnBoard( row, col ) );
    Assert( IsOurPiece( storage, row, col ) );
    UInt32 possibilities = storage->board[row][col].possibilities;
    return ( (possibilities & (possibilities-1)) == 0 );
}

CountAdjacentUnknownPieces
static int CountAdjacentUnknownPieces( StoragePtr storage,
                                       PlayerColor color, int row, int col )
{
    int d;
    int unknowns = 0;

    for ( d = 0; d < 4; d++ ) {
        int r = row + dR[d];
        int c = col + dC[d];

        if ( IsOnBoard( r, c ) && IsColorPiece( storage, r, c, color )
            && IsUnknownPiece( storage, r, c ) ) {
            unknowns++;
        }
    }

    return unknowns;
}

static char *defend_spy_table = "RARROA00RARRRARRXAXA0A00XAXAXAXA";
// Run/Attack/Other/Nothing, >1 unknown:other:danger:moved

LowlyCanAttack
static Boolean LowlyCanAttack( StoragePtr storage, int row,
                              int col, int *otherRow, int *otherCol )
{
    for ( int d = 0; d < 4; d++ ) {
        int r = row + dR[d];
        int c = col + dC[d];

        if ( IsOnBoard( r, c )
            && IsOurPiece( storage, r, c )
            && IsLowlyPiece( storage, r, c ) ) {
            *otherRow = r;
            *otherCol = c;
        }
    }
}

```

```

        return true;
    }
}
return false;
}

UpdateDangerPossibilities
static void UpdateDangerPossibilities( StoragePtr storage )
{
    int row, col;

    for ( row = 0; row < kBoardSize; row++ ) {
        for( col = 0; col < kBoardSize; col++ ) {
            storage->dangers[row][col] = 0;
            storage->known_dangers[row][col] = 0;
        }
    }
    for ( row = 0; row < kBoardSize; row++ ) {
        for( col = 0; col < kBoardSize; col++ ) {
            if ( IsTheirPiece( storage, row, col ) ) {
                UInt32 possibilities =
                    (storage->board[row][col].possibilities &
                     ~kStationaryBits);
                UInt32 known_possibilities = 0;

                if ( storage->board[row][col].rank != kUnknown ) {
                    known_possibilities = possibilities;
                }

                for ( int d = 0; d < 4; d++ ) {
                    int r = row + dR[d];
                    int c = col + dC[d];

                    if ( IsOnBoard( r, c ) ) {
                        storage->dangers[r][c] |= possibilities;
                        storage->known_dangers[r][c] |= known_possibilities;
                    }
                }
            }
        }
    }
}

GetDangerPossibilities
static UInt32 GetDangerPossibilities( StoragePtr storage,
                                     int row, int col )
{
    Assert( IsOnBoard( row, col ) );
    return storage->dangers[row][col];
}

PossibilitiesCouldKill
static Boolean PossibilitiesCouldKill( PieceRank rank,
                                     UInt32 possibilities )
{
    if ( (possibilities & ~kStationaryBits) == 0 ) {
        return false;
    }

    switch ( rank ) {
        case kFlag:
            return true;
        case kBomb:
            return (possibilities & (1 << kMiner)) != 0;
        case kMarshall:
            return (possibilities & ((1 << kMarshall) + (1<< kSpy)))
                != 0;
        default:
            return (possibilities & ((1 << (rank+1)) - 1)) != 0;
    }
}

PossibilitiesCouldKillSafely

```

```

static Boolean PossibilitiesCouldKillSafely( PieceRank rank,
                                           UInt32 possibilities )
{
    if ( (possibilities & ~kStationaryBits) == 0 ) {
        return false;
    }

    switch ( rank ) {
        case kFlag:
            return true;
        case kBomb:
            return (possibilities & (1 << kMiner)) != 0;
        case kMarshall:
            return (possibilities & ((1<< kSpy))) != 0;
        default:
            return (possibilities & ((1 << rank) - 1)) != 0;
    }
}

WillKillPossibilities
static Boolean WillKillPossibilities( PieceRank rank,
                                     UInt32 possibilities )
{
    Assert( possibilities != 0 );

    switch ( rank ) {
        case kFlag:
            return false;
        case kBomb:
            return false;
        case kMiner:
            return (possibilities & ~((1 << kScout) + (1 << kBomb) +
                                     (1 << kFlag))) == 0;
        case kSpy:
            return (possibilities & ~(1 << kMarshall)) == 0;
        default:
            return (possibilities & (((1 << (rank + 1)) - 1) +
                                     (1 << kBomb))) == 0;
    }
}

WillKillOrSuicidePossibilities
static Boolean WillKillOrSuicidePossibilities( PieceRank rank,
                                              UInt32 possibilities )
{
    Assert( possibilities != 0 );

    switch ( rank ) {
        case kFlag:
            return false;
        case kBomb:
            return false;
        case kMiner:
            return (possibilities & ~((1 << kScout) + (1 << kMiner) +
                                     (1 << kBomb) + (1 << kFlag))) == 0;
        case kSpy:
            return (possibilities & ~((1 << kMarshall) + (1 << kSpy)))
                    == 0;
        default:
            return (possibilities & (((1 << rank) - 1) + (1 << kBomb)))
                    == 0;
    }
}

WillPossibilitiesKill
static Boolean WillPossibilitiesKill( UInt32 possibilities,
                                    PieceRank rank )
{
    Assert( possibilities != 0 );
    possibilities &= ~kStationaryBits;
    if ( possibilities == 0 ) {
        return false;
    }
}

```

```

switch ( rank ) {
    case kFlag:
        return true;
    case kBomb:
        return possibilities == (1 << kMiner);
    default:
        return (possibilities & ~((1 << (rank+1))-1)) == 0;
}
}

FindSafeSquare
static Boolean FindSafeSquare( StoragePtr storage, int row,
                             int col, int *safeRow, int *safeCol )
{
    Assert( IsOnBoard( row, col ) );

    PieceRank rank = storage->board[row][col].rank;
    int doff = (storage->playerColor == kBlue ? 0 : 2);
    // Try backwards first

    for ( int d = 0; d < 4; d++ ) {
        int dr = dR[(d + doff) % 4];
        int dc = dC[(d + doff) % 4];
        int r = row + dr;
        int c = col + dc;

        while ( IsOnBoard( r, c ) &&
                IsEmptySquare( storage, r, c ) ) {
            if ( !PossibilitiesCouldKill( rank,
                GetDangerPossibilities( storage, r, c ) ) ) {
                *safeRow = r;
                *safeCol = c;
                return true;
            }
            if ( rank != kScout ) {
                break;
            }
            r += dr;
            c += dc;
        }
    }
    return false;
}

CountEnemies
static void CountEnemies( StoragePtr storage, int row, int col,
                         int *knowns, int *unknowns )
{
    *knowns = 0;
    *unknowns = 0;

    for ( int d = 0; d < 4; d++ ) {
        int r = row + dR[d];
        int c = col + dC[d];

        if ( IsOnBoard( r, c ) && IsTheirPiece( storage, r, c ) ) {
            if ( storage->board[r][c].rank == kUnknown ) {
                *unknowns += 1;
            } else {
                *knowns += 1;
            }
        }
    }
}

/*
static Boolean CanRun( StoragePtr storage, int row, int col, int *runRow, int *runCol )
{
    for ( int d = 0; d < 4; d++ ) {
        int r = row +
            dR[(d + (storage->playerColor == kBlue ? 0 : 2)) % 4];
        // Try backwards first

```

```

int c = col +
    dC[(d + (storage->playerColor == kBlue ? 0 : 2)) % 4];

if ( IsOnBoard( r, c ) &&
    (storage->board[r][c].rank == kEmpty) ) {
    *runRow = r;
    *runCol = c;
    return true;
}
}
return false;
}
*/

FindSafePath
static Boolean FindSafePath( StoragePtr storage,
    Boolean very_safe, Boolean suicide_ok, int from_row,
    int from_col, int to_row, int to_col, int *best_path,
    int *first_row, int *first_col )
{
    Assert( IsOurPiece( storage, from_row, from_col ) );

    PieceRank rank = storage->board[from_row][from_col].rank;
    BoardPossibilities *dangers =
        very_safe ? &storage->dangers : &storage->known_dangers;

    if ( abs( from_row - to_row ) +
        abs( from_col - to_col ) > *best_path ) {
        return false;
    }

    if ( abs( from_row - to_row ) +
        abs( from_col - to_col ) == 1 ) {
        *best_path = 0;
        *first_row = to_row;
        *first_col = to_col;
        return true;
    }

    int path_length_to[kBoardSize][kBoardSize];
    PiecePosition que[kBoardSize * kBoardSize];
    int que_start = 0;
    int que_fin = 0;
    int que_next_len = 0;
    int current_len = 0;
    int row, col;

    for ( row = 0; row < kBoardSize; row++ ) {
        for( col = 0; col < kBoardSize; col++ ) {
            path_length_to[row][col] = -1;
        }
    }

    que[que_fin].row = from_row;
    que[que_fin].col = from_col;
    path_length_to[from_row][from_col] = 0;
    que_fin++;
    que_next_len = que_fin;

    while ( que_fin > que_start ) {
        row = que[que_start].row;
        col = que[que_start].col;
        que_start++;

        for ( int d = 0; d < 4; d++ ) {
            int dr = dR[d];
            int dc = dC[d];
            // scout moves NYI
            int r = row + dr;
            int c = col + dc;

            if ( IsOnBoard( r, c ) && path_length_to[r][c] == -1
                && IsEmptySquare( storage, r, c ) ) {

```

```

    if ( suicide_ok ?
        !PossibilitiesCouldKillSafely( rank, (*dangers)[r][c] )
        : !PossibilitiesCouldKill( rank, (*dangers)[r][c] ) ) {
        path_length_to[r][c] = current_len + 1;
        if ( abs( to_row - r ) + abs( to_col - c ) == 1 ) {
            *best_path = current_len + 1;
            while ( current_len > 0 ) {
                for ( int d = 0; d < 4; d++ ) {
                    int backr = r + dR[d];
                    int backc = c + dC[d];

                    if ( path_length_to[backr][backc] == current_len ) {
                        r = backr;
                        c = backc;
                        break;
                    }
                }
                current_len--;
            }
            *first_row = r;
            *first_col = c;
            return true;
        }
        que[que_fin].row = r;
        que[que_fin].col = c;
        que_fin++;
    } else {
        path_length_to[r][c] = 1000; // Cant go here
    }
}
}

if ( que_start == que_next_len ) {
    que_next_len = que_fin;
    current_len++;
}

return false;
}

```

CalcBoardCRC

```

static UInt32 CalcBoardCRC( StoragePtr storage,
    int from_row, int from_col, int to_row, int to_col )
{
    Assert( !IsOnBoard( from_row, from_col ) ||
        IsOurPiece( storage, from_row, from_col ) );
    Assert( !IsOnBoard( to_row, to_col ) ||
        IsEmptySquare( storage, to_row, to_col ) );

    UInt32 result = 0;

    int row, col;
    int rankish;

    for ( row = 0; row < kBoardSize; row++ ) {
        for ( col = 0; col < kBoardSize; col++ ) {
            if ( row == from_row && col == from_col ) {
                rankish = 0;
            } else if ( row == to_row && col == to_col ) {
                rankish = storage->board[from_row][from_col].rank;
            } else if ( IsEmptySquare( storage, row, col ) ||
                IsWaterSquare( storage, row, col ) ) {
                rankish = 0;
            } else if ( IsOurPiece( storage, row, col ) ) {
                rankish = storage->board[row][col].rank;
            } else {
                rankish = storage->board[row][col].rank + kAddForRankish;
            }
            result += rankish; // Hmm, not a very good CRC
            result = result * 11 + (result >> 25);
        }
    }
}

```

```

    return result;
}

OKMove
static Boolean OKMove( StoragePtr storage, int from_row, int from_col,
int to_row, int to_col )
{
    if ( IsTheirPiece( storage, to_row, to_col ) ) {
        return true;
    }

    UInt32 crc = CalcBoardCRC( storage, from_row, from_col,
                                to_row, to_col );

    long i;
    for ( i = 0; i < storage->repeated_board_count; i++ ) {
        if ( crc == storage->repeated_board[i] ) {
            return false;
        }
    }
    return true;
}

AppendRepeatedBoard
static void AppendRepeatedBoard( StoragePtr storage )
{
    UInt32 crc = CalcBoardCRC( storage, -1, -1, -1, -1 );

    if ( storage->repeated_board_count == kRepeatedBoards ) {
        storage->repeated_board_count--;
        BlockMoveData( &storage->repeated_board[1],
                        &storage->repeated_board[0],
                        storage->repeated_board_count *
                        sizeof(storage->repeated_board[0]) );
    }
    storage->repeated_board[storage->repeated_board_count++] = crc;
}

#ifdef DEBUG_RULES
#define RETURN( x ) DebugStr( x ";" ); return
#else
#define RETURN( x ) return
#endif

FigureOutOurMove
static void FigureOutOurMove( StoragePtr storage,
                             PiecePosition *moveFrom, PiecePosition *moveTo )
{
    int ourRow, ourCol, theirRow, theirCol, row, col, runRow, runCol;
    int rowFirst = storage->playerColor == kRed ?
        0 : kBoardSize - 1;
    int rowLast = storage->playerColor == kRed ?
        kBoardSize - 1 : 0;
    int rowAdd = storage->playerColor == kRed ? 1 : -1;
    int bestUnknowns;
    int bestPath;
    int thisPath;

    UpdateDangerPossibilities( storage );

// USE SPY
    if ( FindPiece( storage, storage->theirColor, kMarshall,
                    &theirRow, &theirCol )
        && FindPiece( storage, storage->playerColor, kSpy,
                    &ourRow, &ourCol )
        && abs( theirRow - ourRow ) +
            abs( theirCol - ourCol ) == 1 ) {
        moveFrom->row = ourRow;
        moveFrom->col = ourCol;
        moveTo->row = theirRow;
        moveTo->col = theirCol;
        RETURN( "\\pUSE SPY" );
    }
}

```

```

// DEFEND AGAINST SPY
if (storage->their_pieces[kSpy] > 0) {
    if ( FindPiece( storage, storage->playerColor, kMarshall,
        &ourRow, &ourCol ) ) {
        int unknowns = CountAdjacentUnknownPieces( storage,
            storage->theirColor, ourRow, ourCol );

        if ( unknowns ) {
            char todo = 0; // R = Run, A = Attack, 0 = Attack with Other
            int base_index = 0;
            Boolean canrun = FindSafeSquare( storage, ourRow, ourCol,
                &runRow, &runCol );

            if ( !canrun ) {
                base_index += 16;
            }
            if ( unknowns > 1 ) {
                base_index += 8;
            }

            for ( int d = 0; d < 4; d++ ) {
                int r = ourRow + dR[d];
                int c = ourCol + dC[d];
                int otherRow, otherCol;

                if ( IsOnBoard( r, c )
                    && IsTheirPiece( storage, r, c )
                    && IsUnknownPiece( storage, r, c ) ) {
                    int index = base_index;
                    if ( LowlyCanAttack( storage, r, c,
                        &otherRow, &otherCol ) ) {
                        index += 4;
                    }
                    if ( CountAdjacentUnknownPieces( storage,
                        storage->theirColor, r, c ) > 0 ) {
                        index += 2;
                    }
                    if ( IsMovedPiece( storage, r, c ) ) {
                        index += 1;
                    }

                    if ( defend_spy_table[index] == 'A' ) { // Attack
                        moveFrom->row = ourRow;
                        moveFrom->col = ourCol;
                        moveTo->row = r;
                        moveTo->col = c;
                        RETURN( "\pDEFEND AGAINST SPY 1" );
                    } else if ( defend_spy_table[index] == '0' ) { // Attack
                        moveFrom->row = otherRow;
                        moveFrom->col = otherCol;
                        moveTo->row = r;
                        moveTo->col = c;
                        RETURN( "\pDEFEND AGAINST SPY 2" );
                    }
                }
            }

            if ( canrun && OKMove( storage, ourRow, ourCol,
                runRow, runCol ) ) {

                moveFrom->row = ourRow;
                moveFrom->col = ourCol;
                moveTo->row = runRow;
                moveTo->col = runCol;
                RETURN( "\pDEFEND AGAINST SPY 3" );
            }
            // Give up! Next rule...
        }
    }
}

// ATTACK WEAKER
for ( row = rowFirst; 0 <= row && row < kBoardSize;
    row += rowAdd ) {
    for( col = 0; col < kBoardSize; col++ ) {

```

```

if ( IsTheirPiece( storage, row, col ) ) {
    UInt32 enemy = storage->board[row][col].possibilities;
    UInt32 danger = GetDangerPossibilities( storage,
                                             row, col );

    int bestDir = -1;
    Boolean isBestRevealed = true;
    PieceRank bestRank = kUnknown;

    for ( int d = 0; d < 4; d++ ) {
        int r = row + dR[d];
        int c = col + dC[d];

        if ( IsOnBoard( r, c ) && IsOurPiece( storage, r, c ) ) {
            if ( !PossibilitiesCouldKill(
                storage->board[r][c].rank, danger ) ) {
                if ( WillKillPossibilities(
                    storage->board[r][c].rank, enemy ) ) {
                    Boolean thisRevealed =
                        IsRevealedPiece( storage, r, c );
                    if ( isBestRevealed || !thisRevealed ) {
                        if ( bestDir == -1 ||
                            (storage->board[r][c].rank > bestRank) ) {
                            bestDir = d;
                            bestRank = storage->board[r][c].rank;
                            isBestRevealed = thisRevealed;
                        }
                    }
                }
            }
        }
    }
}

if ( bestDir != -1 ) {
    moveFrom->row = row + dR[bestDir];
    moveFrom->col = col + dC[bestDir];
    moveTo->row = row;
    moveTo->col = col;
    RETURN( "\pATTACK WEAKER" );
}
}
}
}

// EXPLORE ATTACK
for ( int rnk = kScout; rnk >= kMarshall; rnk-- ) {
    PieceRank rank = (PieceRank) rnk;
    if ( IsLowlyRank( rank ) ) {

        for ( row = rowLast; 0 <= row && row < kBoardSize;
              row -= rowAdd ) {
            for( col = 0; col < kBoardSize; col++ ) {
                if ( IsOurPiece( storage, row, col )
                    && IsRankPiece( storage, row, col, rank ) ) {

                    for ( int d = 0; d < 4; d++ ) {
                        int r = row + dR[d];
                        int c = col + dC[d];

                        if ( IsOnBoard( r, c )
                            && IsTheirPiece( storage, r, c )
                            && IsRankPiece( storage, r, c, kUnknown ) ) {
                            moveFrom->row = row;
                            moveFrom->col = col;
                            moveTo->row = r;
                            moveTo->col = c;
                            RETURN( "\pEXPLORE ATTACK" );
                        }
                    }
                }
            }
        }
    }
}
}
}

```

```

// RETREAT
for ( row = rowLast; 0 <= row && row < kBoardSize;
      row -= rowAdd ) {
    for( col = 0; col < kBoardSize; col++ ) {
        if ( IsOurPiece( storage, row, col )
            && IsMovedPiece( storage, row, col ) ) {

            for ( int d = 0; d < 4; d++ ) {
                int r = row + dR[d];
                int c = col + dC[d];

                if ( IsOnBoard( r, c )
                    && IsTheirPiece( storage, r, c )
                    && WillPossibilitiesKill(
                        storage->board[r][c].possibilities,
                        storage->board[row][col].rank ) ) {
                    bestPath = 1000;
                    for ( int to_row = rowLast; 0 <= to_row &&
                          to_row < kBoardSize; to_row -= rowAdd ) {
                        for( int to_col = 0; to_col < kBoardSize; to_col++ ) {
                            thisPath = bestPath;
                            if ( IsTheirPiece( storage, to_row, to_col )
                                && (IsRankPiece( storage, to_row, to_col,
                                                    kUnknown )
                                   || WillKillPossibilities(
                                       storage->board[row][col].rank,
                                       storage->board[to_row][to_col].possibilities ))
                                && FindSafePath( storage, false, true, row, col,
                                                  to_row, to_col, &thisPath, &runRow, &runCol )
                                && OKMove( storage, row, col, runRow, runCol ) ) {
                                bestPath = thisPath;
                                moveFrom->row = row;
                                moveFrom->col = col;
                                moveTo->row = runRow;
                                moveTo->col = runCol;
                            }
                        }
                    }
                    if ( bestPath < 1000 ) {
                        RETURN( "\pRETREAT" );
                    }
                }
            }
        }
    }
}

```

```

// SCOUT
bestUnknowns = 0;

for ( row = rowLast; 0 <= row && row < kBoardSize;
      row -= rowAdd ) {
    for( col = 0; col < kBoardSize; col++ ) {
        if ( IsOurPiece( storage, row, col )
            && IsRankPiece( storage, row, col, kScout ) ) {
            for ( int d = 0; d < 4; d++ ) {
                int r = row + dR[d];
                int c = col + dC[d];

                while ( IsOnBoard( r, c ) &&
                       IsEmptySquare( storage, r, c ) ) {

                    int knowns, unknowns;
                    CountEnemies( storage, r, c, &knowns, &unknowns );
                    if ( knowns == 0 && unknowns > bestUnknowns &&
                        OKMove( storage, row, col, r, c ) ) {
                        bestUnknowns = unknowns;
                        ourRow = row;
                        ourCol = col;
                        runRow = r;
                        runCol = c;
                    }
                }
            }
        }
    }
}

```

```

        r += dR[d];
        c += dC[d];
    }
}
}
}

if ( bestUnknowns > 0 ) {
    moveFrom->row = ourRow;
    moveFrom->col = ourCol;
    moveTo->row = runRow;
    moveTo->col = runCol;
    RETURN( "\\pSCOUT" );
}

// ATTACK DISTANT

bestPath = 1000;

for ( row = rowFirst; 0 <= row && row < kBoardSize;
      row += rowAdd ) {
    for( col = 0; col < kBoardSize; col++ ) {
        if ( IsTheirPiece( storage, row, col ) ) {
            UInt32 possibilities =
                storage->board[row][col].possibilities;
            UInt32 danger =
                GetDangerPossibilities( storage, row, col );

            if ( (possibilities & ((1 << kBomb) | (1 << kMarshall))) !=
                  ((1 << kBomb) | (1 << kMarshall)) ) {
                for ( int r = rowFirst; 0 <= r && r < kBoardSize;
                      r += rowAdd ) {
                    for( int c = 0; c < kBoardSize; c++ ) {
                        if ( IsOurPiece( storage, r, c ) ) {
                            if ( WillKillPossibilities(
                                storage->board[r][c].rank, possibilities ) ) {
                                if ( storage->board[r][c].rank >= kCaptain ||
                                    !PossibilitiesCouldKill(
                                        storage->board[r][c].rank, danger ) ) {
                                    thisPath = bestPath;
                                    if ( FindSafePath( storage, true, false, r, c,
                                                            row, col, &thisPath, &runRow, &runCol ) ) {
                                        if ( OKMove( storage, r, c, runRow, runCol ) ) {
                                            bestPath = thisPath;
                                            moveFrom->row = r;
                                            moveFrom->col = c;
                                            moveTo->row = runRow;
                                            moveTo->col = runCol;
                                        }
                                    }
                                }
                            }
                        }
                    }
                }
            }
        }
    }
}

if ( bestPath < 1000 ) {
    RETURN( "\\pATTACK DISTANT" );
}

// EXPLORE DISTANT

bestPath = 1000;

for ( row = rowFirst; 0 <= row && row < kBoardSize;
      row += rowAdd ) {
    for( col = 0; col < kBoardSize; col++ ) {

```

```

    if ( IsTheirPiece( storage, row, col ) &&
        storage->board[row][col].rank == kUnknown ) {

        for ( int r = rowFirst; 0 <= r && r < kBoardSize;
              r += rowAdd ) {
            for( int c = 0; c < kBoardSize; c++ ) {
                if ( IsOurPiece( storage, r, c ) &&
                    IsLowlyPiece( storage, r, c ) ) {
                    thisPath = bestPath;
                    if ( FindSafePath( storage, false, true, r, c,
                                         row, col, &thisPath, &runRow, &runCol ) ) {
                        if ( OKMove( storage, r, c, runRow, runCol ) ) {
                            bestPath = thisPath;
                            moveFrom->row = r;
                            moveFrom->col = c;
                            moveTo->row = runRow;
                            moveTo->col = runCol;
                        }
                    }
                }
            }
        }
    }

    if ( bestPath < 1000 ) {
        RETURN( "\\pEXPLORE DISTANT" );
    }

// ATTACK KNOWN WITH SAME DISTANT

    bestPath = 1000;

    for ( row = rowFirst; 0 <= row && row < kBoardSize;
          row += rowAdd ) {
        for( col = 0; col < kBoardSize; col++ ) {
            if ( IsTheirPiece( storage, row, col ) ) {
                UInt32 possibilities =
                    storage->board[row][col].possibilities;

                if ( (possibilities & ((1 << kBomb) | (1 << kMarshall))) !=
                    ((1 << kBomb) | (1 << kMarshall)) ) {
                    for ( int r = rowFirst; 0 <= r && r < kBoardSize;
                          r += rowAdd ) {
                        for( int c = 0; c < kBoardSize; c++ ) {
                            if ( IsOurPiece( storage, r, c ) ) {
                                if ( WillKillOrSuicidePossibilities(
                                    storage->board[r][c].rank, possibilities ) ) {
                                    thisPath = bestPath;
                                    if ( FindSafePath( storage, true, true, r, c,
                                                         row, col, &thisPath, &runRow, &runCol ) ) {
                                        if ( OKMove( storage, r, c, runRow, runCol ) ) {
                                            bestPath = thisPath;
                                            moveFrom->row = r;
                                            moveFrom->col = c;
                                            moveTo->row = runRow;
                                            moveTo->col = runCol;
                                        }
                                    }
                                }
                            }
                        }
                    }
                }
            }
        }
    }

    if ( bestPath < 1000 ) {
        RETURN( "\\pATTACK KNOWN WITH SAME DISTANT" );
    }

```

```

    }

// FIND FLAG
// NYI

// MOVE FORWARD

for ( row = rowLast; 0 <= row && row < kBoardSize; row -= rowAdd ) {
    for( col = 0; col < kBoardSize; col++ ) {
        if ( IsOurPiece( storage, row, col ) ) {
            PieceRank rank = storage->board[row][col].rank;
            if ( rank != kBomb && rank != kFlag ) {
                int r = row + rowAdd;
                if ( IsOnBoard( r, col ) && !IsOurPiece( storage, r, col ) &
                    & OKMove( storage, row, col, r, col ) ) {
                    moveFrom->row = row;
                    moveFrom->col = col;
                    moveTo->row = r;
                    moveTo->col = col;
                    RETURN( "\pMOVE FORWARD" );
                }
            }
        }
    }
}
}
}
}
}

```

```

// MOVE

for ( row = rowLast; 0 <= row && row < kBoardSize;
      row -= rowAdd ) {
    for( col = 0; col < kBoardSize; col++ ) {
        if ( IsOurPiece( storage, row, col ) ) {
            PieceRank rank = storage->board[row][col].rank;
            if ( rank != kBomb && rank != kFlag ) {

                for ( int d = 0; d < 4; d++ ) {
                    int r = row + dR[d];
                    int c = col + dC[d];

                    if ( IsOnBoard( r, c ) &&
                        !IsOurPiece( storage, r, c ) &&
                        OKMove( storage, row, col, r, c ) ) {
                        moveFrom->row = row;
                        moveFrom->col = col;
                        moveTo->row = r;
                        moveTo->col = c;
                        RETURN( "\pMOVE" );
                    }
                }
            }
        }
    }
}
}
}
}
}

```

```

// RESIGN
moveFrom->row = -1;
moveFrom->col = -1;
moveTo->row = -1;
moveTo->col = -1;
RETURN( "\pRESIGN" );
}

```

HandleOurMove

```

static void HandleOurMove( StoragePtr storage,
                          ReportYourMove ReportMove )
{
    PiecePosition moveFrom;
    PiecePosition moveTo;
    Boolean moveStrike;
    MoveResult moveResult;
    UnsignedWide start, finish;
}

```

```

if ( storage->total_time > 10000000 ) { // Time to give up
    // Resign
    moveFrom.row = -1;
    moveFrom.col = -1;
    moveTo.row = -1;
    moveTo.col = -1;
} else {
    FigureOutOurMove( storage, &moveFrom, &moveTo );
}
if ( IsOnBoard( moveTo.row, moveTo.col ) ) {
    moveStrike = storage->board[moveTo.row][moveTo.col].color !=
        kNoColor;
} else {
    moveStrike = false;
}

Microseconds( &start );
(*ReportMove)( &moveFrom, &moveTo, moveStrike, &moveResult );
Microseconds( &finish );
storage->extra_time += finish.lo - start.lo;

if ( moveResult.victory ) {          // We Win! :-)
    storage->victory = true;
} else if ( !moveResult.legalMove ) { // We Lose! :-(!
} else {
    if ( moveStrike ) {
        storage->repeated_board_count = 0;
        Learn( storage, true, moveTo.row, moveTo.col,
            moveResult.rankOfDefender.thePieceRank );
        Learn( storage, false, moveFrom.row, moveFrom.col,
            moveResult.rankOfAttacker.thePieceRank );

        if ( moveResult.attackerRemoved &&
            moveResult.defenderRemoved ) {
            storage->board[moveFrom.row][moveFrom.col] =
                storage->blankSquare;
            storage->board[moveTo.row][moveTo.col] =
                storage->blankSquare;
        } else if ( moveResult.attackerRemoved ) {
            if ( storage->board[moveTo.row][moveTo.col].rank==kBomb ) {
                storage->board[moveFrom.row][moveFrom.col] =
                    storage->blankSquare;
            } else {
                storage->board[moveFrom.row][moveFrom.col] =
                    storage->board[moveTo.row][moveTo.col];
                storage->board[moveTo.row][moveTo.col] =
                    storage->blankSquare;
            }
        } else {
            Assert( moveResult.defenderRemoved );
            storage->board[moveTo.row][moveTo.col] =
                storage->board[moveFrom.row][moveFrom.col];
            storage->board[moveFrom.row][moveFrom.col] =
                storage->blankSquare;
        }
    }

    if ( abs( moveTo.row - moveFrom.row ) +
        abs( moveTo.col - moveFrom.col ) > 1 ) {
        Assert( storage->board[moveFrom.row][moveFrom.col].rank ==
            kScout );
        Learn( storage, false, moveFrom.row, moveFrom.col, kScout );
    } else {
        Learn( storage, false, moveFrom.row, moveFrom.col,
            (PieceRank)kMoved );
    }
    storage->board[moveTo.row][moveTo.col] =
        storage->board[moveFrom.row][moveFrom.col];
    storage->board[moveFrom.row][moveFrom.col] =
        storage->blankSquare;
}
AppendRepeatedBoard( storage );
}

```

```

    AssertValidBoard( storage );
}

MakeAMove
Boolean MakeAMove(
    void *privStorage,          /* 1MB of storage from PositionPieces */
    PlayerColor playerColor,    /* you play red or blue, with red playing first */
    GetOpponentMove *GetMove,   /* callback used to find about opponents last move*/
    ReportYourMove *ReportMove /* callback used to make a move */
)
{
    StoragePtr storage = (StoragePtr) privStorage;
    UnsignedWide start, finish;

    storage->extra_time = 0;
    Microseconds( &start );

    if ( storage->do_getmove ) {
        HandleTheirMove( storage, *GetMove );
    }
    storage->do_getmove = true;

    HandleOurMove( storage, *ReportMove );

    Microseconds( &finish );
    storage->total_time +=
        finish.lo - start.lo - storage->extra_time;

    return storage->victory;
}

```

CHALLENGE.H

```

#ifndef __LL_CHALLENGE__
#define __LL_CHALLENGE__

#ifdef __cplusplus
extern "C" {
#endif

#define kBoardSize 10

typedef enum { kUnknown=0,
    kMarshall=1,kGeneral,kColonel,kMajor,kCaptain,
    kLieutenant,kSergeant,kMiner,kScout,kSpy,
    kBomb,kFlag
} PieceRank;

typedef enum {kRed=1,kBlue=2} PlayerColor;

typedef struct PieceType {
    PieceRank  thePieceRank;    /* rank of a piece */
    PlayerColor thePieceColor;  /* color of a piece */
} PieceType;

typedef PieceType Board[kBoardSize][kBoardSize];
/* Used to provide test code with board configuration. Red starts
   in rows 0..3, Blue starts in rows 6..9 */
/* Squares [4][2], [4][3], [4][6], [4][7] and
   [5][2], [5][3], [5][6], [5][7] are water and cannot
   be occupied */

typedef struct PiecePosition {
    long row; /* 0..9 */
    long col; /* 0..9 */
} PiecePosition;

typedef struct MoveResult {
    PieceType rankOfAttacker;
    /* after a strike, returns identity of attacker */
    PieceType rankOfDefender;
    /* after a strike, returns identity of defender */
}

```

```

Boolean attackerRemoved;
/* true after a strike against a piece of equal or greater rank,
   or against a bomb when the attacker is not a Miner */
Boolean defenderRemoved;
/* true after a strike by a piece of equal or greater rank,
   or against a bomb when the attacker is a Miner,
   or against a Marshall by a Spy */
Boolean victory;
/* true after a strike against the Flag */
Boolean legalMove;
/* true unless you
   - move into an occupied square, or
   - move or strike in a direction other than forward, backward, or sideways, or
   - move more than one square (except Scouts), or
   - move a Bomb or a Flag,
   - move into Water, or
   - strike a square not occupied by an opponent, or
   - make any other illegal move */
} MoveResult;

void PositionPieces(
    void *privStorage,      /* 1MB of preinitialized storage for your use */
    PlayerColor playerColor, /* you play red or blue, with red playing first */
    Board *theBoard         /* provide the initial position of your pieces */
);

typedef void (*ReportYourMove)(
    /* Callback to inform test code of move and
       get results */
    PiecePosition *moveFrom, /* piece you are moving or using to strike */
    PiecePosition *moveTo,   /* destination square or square being struck */
    Boolean strike,          /* false indicates a move, true indicates a strike */
    MoveResult *results      /* returns identity of struck piece and other info */
);

typedef void (*GetOpponentMove)(
    /* Callback to get results of opponents last move */
    PiecePosition *moveFrom, /* piece opponent moved or used to strike */
    PiecePosition *moveTo,   /* destination square or square struck */
    Boolean *strike,         /* false indicates a move, true indicates a strike */
    MoveResult *results      /* returns identity of struck piece and other info */
);

Boolean MakeAMove(
    void *privStorage,      /* 1MB of storage from PositionPieces */
    PlayerColor playerColor, /* you play red or blue, with red playing first */
    GetOpponentMove *GetMove, /* callback used to find about opponents last move */
    ReportYourMove *ReportMove /* callback used to make a move */
);

#ifdef __cplusplus
}
#endif

#endif

```

SPREAD THE WORD: [Slashdot](#) [Digg](#) [Del.icio.us](#) [Reddit](#) [Newsvine](#)

Generate a short URL for this page: [GET LINK](#)